Bedah Sistem & Rekayasa Arsitektur AITF Crawler Engine (Mas Anjas)

Panduan Pemahaman End-to-End, Analisis Mekanisme Anti-Bot, & Rekomendasi Modernisasi untuk Dataset AI (CPT)

Penulis: Naufal Halim | **Lokasi Dokumen:** `KOMDIGI/docs` | **Tanggal:** Oktober 2026

Abstract

Dokumen teknis ini disusun dengan bahasa yang lugas dan mudah dipahami untuk membedah secara tuntas cara kerja **AITF Crawler Engine** (repositori Mas Anjas: `andjsxbp85/crawler-engine`). Fokus kajian ini adalah mengurai secara terperinci arsitektur pipa data dua tahap (*Two-Stage Decoupled Pipeline*), alasan mengapa pengumpulan data Google Search organik lebih unggul daripada sekadar mengandalkan RSS Google News, mekanisme penembusan proteksi anti-bot Google berbasis emulasi display virtual (`Xvfb`), serta alur pembersihan teks untuk menghasilkan dataset *Continual Pre-Training* (CPT) model bahasa alami (LLM). Selain membedah cara kerja saat ini, dokumen ini menyajikan cetak biru rekomendasi rekayasa sistem yang konkret—termasuk paralelisasi asinkron, pembersihan boilerplate otomatis, dan injeksi lapisan stealth—guna meningkatkan throughput ekstraksi data hingga 500%–800%.

Contents

1 Latar Belakang: Mengapa Crawler Ini Dibuat? 3
- 1.1 Keterbatasan RSS Google News . . . 3
- 1.2 Solusi Crawler Mas Anjas . . . 3

2 Filosofi Desain: Mengapa Prosesnya Dipisah Menjadi 2 Tahap? 3
- 2.1 Alasan Teknis Pemisahan Proses . . . 3

3 Bedah Mendalam Tahap 1: Google Search URL Discovery (`aitf-engine`) 5
- 3.1 Bentuk Program & Packaging . . . 5
- 3.2 Konfigurasi Mesin (`default_config.json`) . . . 5
- 3.3 Penjelasan Parameter Kunci . . . 5
- 3.4 Trik Emulasi Display Virtual (`Xvfb`) pada Server Linux . . . 6
- 3.5 Mekanisme Token Lisensi . . . 6
- 3.6 Format Keluaran Tahap 1 . . . 6

4 Bedah Mendalam Tahap 2: Ekstraksi Isi Web (`Scrapper/scraper.py`) 7
- 4.1 Alur Kerja Ekstraksi Konten . . . 7
- 4.2 Algoritma Pencocokan CSS Selector Hierarkis . . . 7
- 4.3 Ekstraksi Teks Bersih via JavaScript In-Browser . . . 7
- 4.4 Pemisahan Data Bersih vs. Log Galat . . . 8
- 4.5 Sifat Konkurensi Tahap 2: Sekuensial Murni . . . 8

5 Analisis Performa, Konkurensi, & Siklus Hidup Memori 9
- 5.1 Karakteristik Eksekusi Hybrid . . . 9
- 5.2 Siklus Hidup Memori (Memory Lifecycle) . . . 9
- 5.3 Hitungan Estimasi Waktu Scraping pada Tahap 2 . . . 9

6 Evaluasi Sistem: Kelebihan & Keterbatasan Saat Ini **9**
6.1 Kelebihan Utama Sistem Mas Anjas 9
6.2 Keterbatasan yang Perlu Dibenahi 10

7 Rekomendasi Rekayasa: Peningkatan & Modernisasi Sistem **11**
7.1 Rekomendasi 1: Paralelisasi Asinkron Tahap 2 11
7.2 Rekomendasi 2: Pre-Resolve Redirect Google Secara Asinkron 12
7.3 Rekomendasi 3: Ekstraksi Konten Cerdas Otomatis (*Boilerplate Removal*) 12
7.4 Rekomendasi 4: Injeksi Skrip Anti-Deteksi di Tahap 2 12
7.5 Rekomendasi 5: Penyatuan Alur Kerja (*Unified Pipeline Orchestrator*) 13
7.6 Rekomendasi 6: Integrasi Hermes Agent (Autonomous Strategy & Topic Expansion) 13
7.6.1 Fungsi Kunci Hermes Agent 13
7.7 Rekomendasi 7: Integrasi JEV TypeSafe AI (Deterministic Quality Gatekeeper) . . . 14
7.7.1 Mengapa Menggunakan JEV Dibandingkan LLM Generatif Biasa? 14
7.8 Rekomendasi 8: Arsitektur Terpadu – The Agentic CPT Harvester Pipeline 15

8 Panduan Praktis Pengoperasian End-to-End Saat Ini **17**
8.1 Langkah di Windows 17
8.2 Langkah di Linux (Ubuntu / Server Tanpa Monitor) 17

9 Kesimpulan **17**

1 Latar Belakang: Mengapa Crawler Ini Dibuat?

Dalam pengembangan model kecerdasan buatan berbasis teks (LLM), ketersediaan data teks berbahasa Indonesia yang berkualitas tinggi adalah kunci utama keberhasilan pelatihan model, khususnya pada fase **Continual Pre-Training (CPT)** dan domain adaptation.

1.1 Keterbatasan RSS Google News

Banyak pengembang awalnya mengandalkan umpan RSS (*Really Simple Syndication*) dari Google News untuk mengumpulkan artikel berita. Namun, pendekatan RSS memiliki sejumlah kelemahan fatal:

1. **Konten Terpotong (Snippet Saja):** RSS Google News hanya menyajikan judul dan satu hingga dua kalimat ringkasan pendek. Teks artikel lengkap tetap harus diambil dengan membuka halaman web aslinya.
2. **Kurasi Terbatas:** RSS hanya menampilkan berita yang diindeks dalam tab Google News dan biasanya hanya memuat puluhan artikel terbaru, bukan seluruh khazanah informasi web.
3. **Tidak Relevan untuk Topik Terfokus:** Untuk kata kunci teknis, pendidikan, kebijakan publik, atau sejarah yang tidak sedang menjadi berita hangat, RSS Google News sering kali tidak membuahkan hasil.

1.2 Solusi Crawler Mas Anjas

Untuk mengatasi keterbatasan tersebut, Mas Anjas merancang **AITF Crawler Engine**. Sistem ini bertindak langsung sebagai perayap mesin pencari Google Search organik:

- Mampu mencari topik apa saja berdasarkan daftar kata kunci (`daftar_keyword.txt`).
- Memanen ratusan URL hasil pencarian Google asli dalam hitungan menit.
- Menyediakan mesin ekstraksi teks untuk mengubah artikel web menjadi format dokumen bersih Markdown yang langsung siap di-tokenisasi untuk pelatihan AI.

2 Filosofi Desain: Mengapa Prosesnya Dipisah Menjadi 2 Tahap?

Salah satu keputusan arsitektur paling penting pada Crawler Mas Anjas adalah memisahkan keseluruhan proses perayapan menjadi dua direktori dan dua program independen:

Pipa Kerja Dua Tahap (Two-Stage Decoupled Pipeline) **TAHAP 1: URL DISCOVERY ENGINE (Direktori: Windows / Linux / MacOS)**
Membuka Google Search ⟶ Emulasi Browser Nyata ⟶ Panen Link Redirect ⟶ `crawler_url.txt`

⇓ *(Berkas teks daftar URL sebagai perantara)*

TAHAP 2: CONTENT EXTRACTION SCRAPER (Direktori: Scrapper)
Buka URL Asli ⟶ Ikuti Redirect 302 ⟶ Ekstrak Teks via CSS Path ⟶ `hasil_scrapping.md`

2.1 Alasan Teknis Pemisahan Proses

1. **Tingkat Sensitivitas Anti-Bot yang Berbeda:**

- *Google Search (Tahap 1)* memiliki perlindungan bot paling ketat di dunia. Google memantau fingerprint browser, kecepatan pencarian, dan apakah browser memiliki layar fisik. Jika terdeteksi bot, IP akan langsung terkena blokir atau tantangan CAPTCHA yang sulit. Oleh karena itu, Tahap 1 membutuhkan perlakuan khusus (*special treatment*) berupa browser non-headless dan emulasi display.
- *Website Target (Tahap 2)* adalah portal berita atau blog biasa. Tingkat proteksinya jauh lebih rendah daripada Google, sehingga proses ekstraksi teks dapat dijalankan dengan Playwright headless standar yang cepat dan ringan.

2. **Pemisahan Risiko & Modularity:** Jika proses scraping artikel di Tahap 2 mengalami error pada satu situs tertentu, proses pencarian Google di Tahap 1 tidak terganggu. Pengguna tidak perlu mengulang pencarian Google dari awal karena daftar URL sudah tersimpan aman di berkas teks `crawler_url.txt`.

3. **Kebersihan Dataset Pelatihan AI:** Tahap 2 dirancang fokus memproduksi file Markdown murni tanpa header boilerplate, metadata tak penting, atau pesan error, sehingga dataset tidak tercemar sampah teks.

3 Bedah Mendalam Tahap 1: Google Search URL Discovery (`aitf-engine`)

Tahap pertama bertugas menerima daftar kata kunci, mencari di Google, dan mengumpulkan semua link hasil pencarian.

3.1 Bentuk Program & Packaging

Program dikemas dalam bentuk biner siap pakai (*standalone binary*) yang dikompilasi menggunakan PyInstaller berbasis Python 3.13:

- `Windows/aitf-engine.exe` (Ukuran ~25 MB)
- `Linux/aitf-engine` (Ukuran ~26 MB)
- `MacOS/aitf-engine` (Ukuran ~23 MB)

Di dalam biner tersebut sudah terintegrasi script pengendali CLI (`cli`), modul perayap Google (`crawler/google_crawler.py`), manajemen kata kunci (`crawler/keywords.py`), serta pustaka Selenium WebDriver.

3.2 Konfigurasi Mesin (`default_config.json`)

Perilaku pencarian dikendalikan oleh berkas JSON yang mudah dikustomisasi:

```
{
  "parallel": 3,
  "parallel_tab_each": 0,
  "keywords": [],
  "keywords_file": "daftar_keyword.txt",
  "keyword_type": "keywords_file",
  "activate_super_mode": false,
  "user_agent_file": "user-agent-list.csv",
  "incognito": true,
  "headless": false,
  "auto_recrawl": true,
  "remote_driver_url": null,
  "search_image_mode": false,
  "getting_url_img_search_minimal": 20,
  "unique_url_output": true
}
```

3.3 Penjelasan Parameter Kunci

- **`"parallel": 3` (Konkurensi Paralel):** Engine secara simultan membuka **3 proses jendela Chrome sekaligus**. Setiap jendela memproses kata kunci yang berbeda dari `daftar_keyword.txt`. Hal ini membuat pencarian banyak topik selesai 3 kali lebih cepat dibandingkan proses sekuensial.
- **`"incognito": true`:** Setiap jendela Chrome dijalankan dalam mode penyamaran (*Incognito*). Hal ini memastikan sesi pencarian bersih dari riwayat, cookie lama, atau data profil yang bisa menghubungkan aktivitas bot dengan identitas tertentu.
- **`"headless": false` (Trik Anti-Bot Utama):** Browser sengaja dibuka dengan antarmuka visual penuh (*non-headless*). Mode headless bawaan Chrome sangat mudah dideteksi oleh skrip pertahanan Google karena tidak memiliki context render grafis lengkap dan memiliki flag JavaScript khusus.
- **`"unique_url_output": true`:** Mengaktifkan deduplikasi otomatis sehingga URL duplikat yang muncul di beberapa halaman pencarian tidak dicatat berulang kali.

3.4 Trik Emulasi Display Virtual (Xvfb) pada Server Linux

Banyak server pengolahan data atau VPS tidak memiliki monitor fisik (hanya baris perintah terminal SSH). Jika Chrome non-headless dijalankan di sana, program biasanya akan langsung crash dengan pesan error `cannot open display`.

Mas Anjas mengatasi masalah ini secara cerdas pada skrip `Linux/linux-installer.sh`:

```
# Install X Virtual Framebuffer (Xvfb)
sudo apt install -y xvfb chromium chromium-driver

# Set alias permanen
alias aitf-engine="xvfb-run -a ./aitf-engine"
```

Perintah `xvfb-run -a` membuat **layar monitor virtual di dalam memori RAM**. Ketika Chrome berjalan, browser mengira ia sedang menampilkan jendela visual ke monitor beresolusi nyata, padahal semuanya hanya berjalan di dalam RAM server. Dengan trik ini, server Linux tanpa layar tetap bisa merayapi Google Search tanpa terdeteksi sebagai bot headless.

3.5 Mekanisme Token Lisensi

Biner `aitf-engine` dilengkapi gerbang autentikasi. Pengguna wajib menyertakan token:

```
./aitf-engine crawl --token=R8trcqTXSWyPPnkIbOI6QKV3dm+2xsGeP90...
# atau set melalui environment variable:
export CRAWLER_ENGINE_TOKEN="R8trcqTXSWyPPnkIbOI6QKV3dm+2xsGeP90..."
```

Jika token tidak valid atau tidak disertakan, biner menolak berjalan dengan pesan: *"Invalid token, please contact administrator"*.

3.6 Format Keluaran Tahap 1

Hasil akhir Tahap 1 disimpan di `Windows/output/crawler_url.txt`. Tautan yang tercatat adalah tautan pengalihan Google (*Google Redirect URL*):

```
url
https://www.google.com/goto?url=CAESawHrOzAVFnfWwRExaPQTEyAbPR8...
https://www.google.com/goto?url=CAESbwHrOzAVS8-s2NFxz3X0dMDT37k...
```

Tautan ini adalah format URL resmi saat pengguna mengeklik hasil pencarian Google, yang nantinya akan dialihkan secara otomatis ke alamat website artikel asli.

4 Bedah Mendalam Tahap 2: Ekstraksi Isi Web (`Scrapper/scraper.py`)

Tahap kedua bertugas menerima daftar URL dari Tahap 1, mengunjungi website tujuan, mengekstrak isi teks berita/artikel, dan menyimpannya menjadi dokumen dataset pelatihan AI yang bersih.

4.1 Alur Kerja Ekstraksi Konten

Script utama pada tahap ini adalah `Scrapper/scraper.py`. Script ini bekerja melalui empat langkah:

1. Membaca daftar URL dari berkas teks (`-urls=Windows/output/crawler_url.txt`).
2. Membaca aturan pemetaan CSS selector dari berkas CSV jika disediakan (`-css-paths=...`).
3. Membuka browser Playwright atau Selenium, menelusuri redirect tautan Google hingga mendarat di website target, lalu mengekstrak teks menggunakan JavaScript DOM evaluation.
4. Menyimpan hasil teks bersih ke file Markdown dan mencatat URL yang bermasalah ke file log terpisah.

4.2 Algoritma Pencocokan CSS Selector Hierarkis

Tantangan terbesar saat mengekstrak teks dari berbagai website adalah: **setiap website memiliki struktur HTML yang berbeda**. Paragraf berita di Kompas berada di tag yang berbeda dengan Detik atau situs pemerintah.

Mas Anjas menerapkan fungsi `get_css_selector_for_url` yang mencari selector terbaik dengan urutan hierarki:

1. **Pencocokan URL Lengkap (*Exact Match*):** Mencocokkan URL secara persis jika ada halaman khusus yang dipetakan.
2. **Pencocokan Domain Penuh (*Full Domain*):** Contohnya `www.kompas.com` dipetakan ke `"div.read__article p"`.
3. **Pencocokan Domain tanpa WWW:** Contohnya `kompas.com`.
4. **Pencocokan Substring:** Mencari apakah nama domain terkandung dalam kunci CSV.
5. **Fallback Default (`"body p"`):** Jika domain tidak terdaftar pada file CSV, scraper otomatis menggunakan selector default `"body p"` (semua paragraf di dalam body) dan menampilkan pesan peringatan di terminal: *"css-path url 'x' tidak didefinisikan, menggunakan default path"*.

4.3 Ekstraksi Teks Bersih via JavaScript In-Browser

Scraper mengeksekusi fungsi JavaScript langsung di dalam konteks halaman browser Playwright:

```
scrapped_text = page.evaluate("""(cssPath) => {
    let elementText = document.querySelectorAll(cssPath);
    let scrapped = '';
    elementText.forEach(el => {
        scrapped = `${scrapped}\\n${el.innerText}`;
    });
    return scrapped;
}""", css_path)
```

Fungsi ini mengambil seluruh elemen yang cocok dengan selector, membaca `innerText`-nya (sehingga tag-tag HTML seperti `<div>`, `<span>`, atau `<script>` otomatis terbuang), dan menggabungkannya menjadi untaian teks bersih.

4.4 Pemisahan Data Bersih vs. Log Galat

Keluaran disimpan menggunakan fungsi `save_output`:

- **Mode Merge (`-output-mode=merge`):** Menggabungkan seluruh teks artikel yang berhasil diekstrak ke dalam satu berkas Markdown tunggal (`hasil_scrapping.md`). Setiap artikel dipisahkan secara rapi menggunakan penanda horizontal murni:

  `\n\n--\n\n`

  Format ini tidak memuat judul buatan atau URL sumber, menjadikannya langsung siap diumpankan ke pipeline tokenisasi LLM (*Continual Pre-Training*).

- **Mode Each (`-output-mode=each`):** Menyimpan tiap artikel ke dalam satu berkas terpisah (misal: `scraped_001_kompas_com.md`).

- **Log Pelaporan Kegagalan (`hasil_scrapping.log`):** URL yang gagal dibuka (timeout, koneksi putus) atau menghasilkan teks kosong sama sekali **tidak dimasukkan ke dalam file Markdown**. Detail kegagalannya dicatat di file log terpisah agar dataset pelatihan tidak tercemar artikel kosong.

4.5 Sifat Konkurensi Tahap 2: Sekuensial Murni

Berbeda dengan Tahap 1 yang membuka 3 browser paralel, pada Tahap 2, fungsi `scrape_with_playwright` menggunakan perulangan tunggal:

```
page = context.new_page()
for idx, raw_url in enumerate(urls, 1):
    # Buka URL satu per satu
    page.goto(url, wait_until="domcontentloaded", timeout=timeout * 1000)
    page.wait_for_timeout(1000) # Jeda statis 1 detik
    # Ekstrak -> Simpan -> Lanjut ke iterasi berikutnya
```

Artinya, Tahap 2 memproses URL secara **sekuensial (satu demi satu)**.

5 Analisis Performa, Konkurensi, & Siklus Hidup Memori

5.1 Karakteristik Eksekusi Hybrid

Crawler Mas Anjas memadukan dua pendekatan kerja yang berbeda antara Tahap 1 dan Tahap 2:

Parameter Evaluasi	Tahap 1: Google Crawl (`aitf-engine`)	Tahap 2: Web Scraper (`scraper.py`)
Model Eksekusi	**Paralel Multi-Process** (`"parallel": 3`)	**Sekuensial Single-Thread** (Loop `for`)
Engine Browser	Selenium + ChromeDriver asli	Playwright Chromium sync (opsional Selenium)
Tampilan Browser	Non-headless (atau `Xvfb` di Linux)	Headless (tanpa tampilan visual)
Konsumsi RAM	Cukup tinggi (∼600MB–1.2GB untuk 3 Chrome)	Rendah dan sangat stabil (∼150MB–250MB)
Throughput Kecepatan	Cepat (banyak keyword diproses bersamaan)	Lambat (∼4–6 detik per URL)
Risiko Deteksi Bot	Tinggi (ditangani via non-headless & incognito)	Rendah (hanya akses halaman berita publik)

Table 1: Komparasi Karakteristik Eksekusi Tahap 1 dan Tahap 2

5.2 Siklus Hidup Memori (Memory Lifecycle)

Karena sistem ini berbasis **Batch CLI** (dijalankan via perintah terminal untuk satu tugas lalu selesai), sistem ini memiliki keunggulan besar dalam manajemen memori:

- Begitu proses selesai, sistem operasi langsung menutup browser Chrome dan **membersihkan RAM 100%**.
- Bebas dari penyakit umum browser scraper yang berjalan 24/7, yaitu kebocoran memori (*memory leak*) di mana browser yang dibiarkan menyala berhari-hari perlahan memakan RAM hingga server kehabisan memori dan crash.

5.3 Hitungan Estimasi Waktu Scraping pada Tahap 2

Karena Tahap 2 berjalan sekuensial dengan jeda 1 detik per halaman dan waktu loading halaman rata-rata 3 detik (total ∼4 detik per URL):

- **92 URL** (seperti pada data log Mas Anjas): Membutuhkan waktu sekitar **∼6 menit**.
- **500 URL**: Membutuhkan waktu sekitar **∼33 menit**.
- **2.000 URL**: Membutuhkan waktu sekitar **∼2,2 jam**.

Angka ini menunjukkan bahwa Tahap 2 adalah area yang paling potensial untuk ditingkatkan kecepatannya melalui paralelisasi.

6 Evaluasi Sistem: Kelebihan & Keterbatasan Saat Ini

6.1 Kelebihan Utama Sistem Mas Anjas

1. **Pemisahan Masalah yang Bersih (*Separation of Concerns*):** Memisahkan risiko pemblokiran Google dari proses ekstraksi isi web adalah strategi cerdas dan terbukti tangguh.

2. **Keluaran Teks Bersih untuk CPT AI:** Format Markdown tanpa header dan isolasi teks error ke berkas log terpisah membuat dataset siap pakai tanpa perlu proses pembersihan manual yang rumit.
3. **Solusi Linux Xvfb yang Elegan:** Memungkinkan server cloud murah tanpa GUI fisik untuk menjalankan Chrome non-headless demi mengelabui sistem deteksi bot Google.
4. **Multi-OS Portable:** Tersedianya biner terkompilasi untuk Windows, Linux, dan macOS mempermudah tim untuk langsung bekerja tanpa pusing mengatur dependensi Python awal.

6.2 Keterbatasan yang Perlu Dibenahi

1. **Tahap 2 Terlalu Lambat (Bottleneck Sekuensial):** Pengambilan ratusan atau ribuan URL artikel memakan waktu lama karena diproses satu per satu secara berurutan.
2. **Fallback Selector `"body p"` Rentan Noise:** Jika domain tidak didaftarkan di CSV, selector `"body p"` sering kali ikut menyerap menu navigasi, tombol share sosial media, dan disclaimer hak cipta di bagian bawah halaman.
3. **Beban Browser Berlebih untuk Link Redirect:** Membuka browser Playwright penuh hanya untuk mengikuti redirect URL Google memakan resource yang tidak perlu.
4. **Belum Ada Lapisan Stealth JavaScript di Tahap 2:** Jika artikel berada di situs media besar yang diproteksi Cloudflare, Playwright standar tanpa modifikasi objek navigator dapat terkena blokir.
5. **Dua Perintah Manual Terpisah:** Pengguna harus mengetikkan perintah Tahap 1, menunggu sampai selesai, lalu mengetikkan perintah Tahap 2 secara manual.

7 Rekomendasi Rekayasa: Peningkatan & Modernisasi Sistem

Berikut adalah 6 rekomendasi rekayasa sistem konkret untuk menyempurnakan AITF Crawler Engine menjadi mesin pengumpul dataset AI kelas produksi yang sangat cepat dan tangguh:

Cetak Biru Peningkatan Arsitektur (Engineering Blueprint)

1. **Paralelisasi Asinkron Tahap 2 (`async_playwright` + Semaphore):** Memangkas durasi ekstraksi hingga 80% (dari 6 menit menjadi kurang dari 1 menit untuk 100 URL).
2. **Penyelesaian Redirect Google Ringan (*Pre-Resolution*):** Mengikuti tautan redirect `google.com/goto?url=...` menggunakan HTTP client ringan sebelum memanggil browser.
3. **Ekstraksi Konten Cerdas Otomatis (*Boilerplate Removal*):** Mengintegrasikan library seperti `trafilatura` sebagai pengganti fallback `"body p"`.
4. **Injeksi Skrip Anti-Deteksi di Tahap 2:** Menyamarkan properti navigator agar Playwright tidak diblokir situs berita berproteksi Cloudflare.
5. **Penyatuan Alur Kerja Otomatis (*Unified Pipeline Chaining*):** Script orchestrator untuk menjalankan Tahap 1 dan Tahap 2 dalam satu perintah end-to-end.
6. **Dukungan Format Dataset AI Modern (`JSONL` & Parquet):** Selain Markdown, sediakan opsi ekspor berstruktur untuk pipeline LLM modern.

7.1 Rekomendasi 1: Paralelisasi Asinkron Tahap 2

Alih-alih memproses satu per satu dengan `sync_playwright`, ubah ke `async_playwright` dengan membatasi jumlah tab bersamaan menggunakan `asyncio.Semaphore(8)`:

```
import asyncio
from playwright.async_api import async_playwright

async def scrape_single_url(context, url, css_path, sem, timeout=30):
    async with sem:  # Batasi maksimal 8 tab bersamaan
        page = await context.new_page()
        try:
            await page.goto(url, wait_until="domcontentloaded",
                timeout=timeout*1000)
            await page.wait_for_timeout(500)
            text = await page.evaluate("""(cssPath) => {
                let els = document.querySelectorAll(cssPath);
                let out = '';
                els.forEach(el => { out += '\\n' + el.innerText; });
                return out;
            }""", css_path)
            return {"url": url, "content": text.strip(), "status":
                "SUCCESS"}
        except Exception as e:
            return {"url": url, "content": "", "status": f"ERROR:
                {str(e)}"}
        finally:
            await page.close()

async def main_scrape_parallel(urls, concurrency=8):
    sem = asyncio.Semaphore(concurrency)
    async with async_playwright() as p:
```

```
        browser = await p.chromium.launch(headless=True)
        context = await browser.new_context()
        tasks = [scrape_single_url(context, u, "body p", sem) for u in
            urls]
        results = await asyncio.gather(*tasks)
        await browser.close()
        return results
```

7.2 Rekomendasi 2: Pre-Resolve Redirect Google Secara Asinkron

Tautan dari Tahap 1 berbentuk:

`https://www.google.com/goto?url=CAES...`

Membuka browser Playwright penuh hanya untuk menunggu Google melakukan HTTP 302 redirect sangat membuang CPU dan RAM.

Gunakan client HTTP asinkron (**httpx**) untuk menelusuri alamat aslinya terlebih dahulu dalam hitungan milidetik:

```
import httpx

async def resolve_google_redirect(goto_url: str) -> str:
    async with httpx.AsyncClient(follow_redirects=True, timeout=10.0) as
        client:
        try:
            resp = await client.head(goto_url)
            return str(resp.url) # URL domain artikel yang sesungguhnya
        except Exception:
            return goto_url
```

Dengan men-resolve URL di awal, scraper dapat langsung membuka alamat portal berita tanpa perantara Google.

7.3 Rekomendasi 3: Ekstraksi Konten Cerdas Otomatis (*Boilerplate Removal*)

Saat ini, jika situs baru tidak didaftarkan di `sample_css_paths.csv`, scraper beralih ke `"body p"`. Akibatnya, menu navigasi, teks widget samping, dan disclaimer copyright ikut terserap.

Solusinya adalah mengintegrasikan library ekstraksi artikel standar industri AI seperti **trafilatura**:

```
import trafilatura

def extract_clean_article(html_content: str) -> str:
    # Trafilatura otomatis membuang menu, header, iklan, dan footer
    extracted_text = trafilatura.extract(
        html_content,
        include_comments=False,
        include_tables=True,
        no_fallback=False
    )
    return extracted_text or ""
```

Jika dipadukan, scraper cukup mengambil HTML halaman penuh, lalu membiarkan algoritma cerdas memilah mana badan artikel berita utama dan mana bagian sampah navigasi.

7.4 Rekomendasi 4: Injeksi Skrip Anti-Deteksi di Tahap 2

Banyak portal berita nasional kini diproteksi oleh Cloudflare. Agar browser Playwright di Tahap 2 tidak dicegat halaman Cloudflare Challenge, tambahkan skrip penyamaran (*stealth script*) pada context Playwright:

```
context = await browser.new_context(
    user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64)
       AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0
       Safari/537.36"
)
await context.add_init_script("""
    Object.defineProperty(navigator, 'webdriver', {get: () => undefined});
    Object.defineProperty(navigator, 'plugins', {get: () => [1, 2, 3, 4,
       5]});
    window.chrome = { runtime: {} };
""")
```

7.5 Rekomendasi 5: Penyatuan Alur Kerja (*Unified Pipeline Orchestrator*)

Buat satu skrip pembungkus terpadu (misal `run_pipeline.py`) sehingga pengguna cukup menjalankan satu perintah:

```
python run_pipeline.py --keywords="Sekolah Rakyat, Matematika Aljabar"
   --output="dataset_cpt.md" --concurrency=8
```

Skrip pembungkus ini akan secara otomatis:

(a) Menjalankan biner `aitf-engine crawl` untuk mengumpulkan tautan Google.

(b) Mengonfirmasi berkas `crawler_url.txt` telah terbentuk.

(c) Meluncurkan scraper paralel untuk menghasilkan dataset Markdown bersih.

(d) Menampilkan ringkasan: *"Berhasil mengekstrak 184 artikel (4,2 MB teks bersih CPT) dalam 1 menit 12 detik"*.

7.6 Rekomendasi 6: Integrasi Hermes Agent (Autonomous Strategy & Topic Expansion)

Diadopsi dari arsitektur **CommunitEyes Pro SNA** (`hermes_browser_agent.py` dan `hermes_controller.py`), **Hermes Agent** dapat diintegrasikan sebagai lapisan orkestrator strategis di hulu pipa perayapan (*upstream strategic layer*).

Peran Hermes Agent dalam Ekosistem Crawler Mas Anjas [**Domain CPT Target**] ⟶ **Hermes Strategy Agent** (Gemini / Claude / Step-Flash)
↓ Sintesis Otomatis Query Google & Operator Dorking Cerdas
[**daftar_keyword.txt Dinamis**] ⟶ `aitf-engine crawl` (Tahap 1)
↓ Adaptive Concurrency & Session Guard
[**Mitigasi IP Ban Google**] ⟵ Hermes mengatur ritme jeda batch secara cerdas

7.6.1 Fungsi Kunci Hermes Agent

1. **Ekspansi Kata Kunci Cerdas (*Dynamic Topic Expansion*):** Alih-alih mengisi `daftar_keyword.txt` secara manual dengan 3 kata kunci statis, pengguna cukup memberikan satu tema besar (misalnya: *"Transformasi Digital Layanan Publik"*). Hermes secara otonom memecah tema tersebut menjadi 50 variasi kata kunci turunan yang spesifik dan belum pernah dirayapi sebelumnya.
2. **Sintesis Query Google Dorking:** Hermes mengombinasikan kata kunci dengan operator pencarian lanjutan untuk menyasar dokumen berkualitas tinggi:

- `site:go.id "Sekolah Rakyat" filetype:pdf` (menyasar dokumen regulasi resmi pemerintah).
- `intitle:"analisis data" "kebijakan publik" after:2025` (menyasar kajian komparatif terkini).

3. **Adaptive Rate-Limiting Controller:** Memantau frekuensi respons dari Google Search. Jika mulai terdeteksi gejala perlambatan respons, Hermes secara otomatis menginstruksikan Tahap 1 untuk melambatkan jeda antar-batch atau merotasi User-Agent.

```python
def call_hermes_for_crawler_strategy(domain_target: str, count: int = 20)
    -> list[str]:
    """Hermes Agent: Merumuskan strategi kata kunci dan Google Dorking."""
    prompt = f"""
    Bertindaklah sebagai Data Strategy Agent untuk Continual Pre-Training
        (CPT) LLM.
    Target domain: {domain_target}.
    Buatkan {count} query pencarian Google optimal dengan operator
        dorking cerdas
    (site:, intitle:, filetype:) untuk memanen artikel teks mendalam dan
        kaya informasi.
    Format keluaran: JSON list string tanpa penjelasan tambahan.
    """
    # Panggilan ke model Hermes (seperti pada modul communiteyes-pro)
    response = execute_hermes_agent(prompt)
    return parse_json_keywords(response)
```

7.7 Rekomendasi 7: Integrasi JEV TypeSafe AI (Deterministic Quality Gatekeeper)

Diadopsi dari modul **Bot KOMDIGI** (`decision/jev.py`), **JEV System One dari TypeSafe AI** (`https://api.typesafe.ai/v1/systemone`) dapat diintegrasikan sebagai **pintu gerbang kendali mutu deterministik (*downstream quality gatekeeper*)** di hilir pipa ekstraksi Tahap 2.

Peran JEV TypeSafe AI sebagai Filter Mutu Dataset CPT [**Teks Kasar Hasil Playwright**]
$\longrightarrow$ **JEV System One Engine** (`POST /v1/systemone`)
$\downarrow$
Operasi JEV:
- **choice** $\longrightarrow$ Klasifikasi: [article, paywall, menu_dump, ad_hub, 404]
- **score** $\longrightarrow$ Evaluasi Kepadatan Informasi $[0.0 - 1.0]$
- **noul** $\longrightarrow$ Filter Biner: Apakah layak masuk dataset CPT? (True/False)

$\downarrow$ Hanya lolos jika: `choice=="article"` & `score >= 0.70` & `noul == True`
[**hasil_scrapping.md**] $\longleftarrow$ Dataset CPT Terverifikasi 100% Bersih Tanpa Sampah

7.7.1 Mengapa Menggunakan JEV Dibandingkan LLM Generatif Biasa?

- **Kontrak Keluaran Terstruktur (*Strict Typed Contract*):** JEV tidak mengembalikan teks bebas (*free-form text*) yang rawan halusinasi, melainkan nilai typed terverifikasi: `choice`, `score`, dan `noul`.
- **Latensi Sangat Cepat & Hemat Biaya:** Dirancang khusus untuk evaluasi status mikro (*micro-decision*), JEV mengevaluasi artikel dalam waktu kurang dari 200 milidetik, jauh lebih cepat dan murah dibandingkan memanggil model LLM generatif besar untuk setiap artikel.
- **Menghilangkan Data Sampah dari Dataset CPT:** Jika selector fallback `"body p"` menyerap teks navigasi menu atau disclaimer hak cipta, JEV langsung mendeteksinya sebagai `menu_dump` dan menolaknya (`noul: False`).

```python
import httpx
from pydantic import BaseModel

async def verify_content_with_jev(url: str, extracted_text: str) -> bool:
    """JEV TypeSafe AI: Gatekeeper kualitas teks artikel untuk dataset
       CPT."""
    payload = {
        "model": "jev-latest",
        "state": {"url": url, "snippet": extracted_text[:2500]},
        "questions": {
            "page_classification": {
                "type": "choice",
                "instructions": "Klasifikasikan struktur teks halaman
                   ini",
                "criteria": {
                    "valid_article": "Teks berupa artikel berita atau
                       informasi utuh yang koheren",
                    "nav_dump": "Teks dominan menu navigasi, footer, atau
                       link daftar isi",
                    "paywall": "Halaman login berbayar atau pembatasan
                       akses",
                    "spam_ad": "Halaman spam iklan, promosi judi, atau
                       konten klikbait"
                }
            },
            "information_density": {
                "type": "score",
                "instructions": "Beri skor kepadatan informasi berbobot
                   edukasi/ilmiah [0.0 - 1.0]"
            },
            "cpt_ready": {
                "type": "noul",
                "instructions": "Apakah teks ini layak masuk ke korpus
                   training CPT LLM?"
            }
        }
    }

    headers = {"Authorization": f"Bearer {TYPESAFE_API_KEY}"}
    async with httpx.AsyncClient() as client:
        resp = await client.post("https://api.typesafe.ai/v1/systemone",
           json=payload, headers=headers)
        ans = resp.json().get("answers", {})

        is_article = ans.get("page_classification", {}).get("choice") ==
           "valid_article"
        quality_score = ans.get("information_density", {}).get("score",
           0.0)
        cpt_ready = ans.get("cpt_ready", {}).get("noul", 0.0) >= 0.5

        # Lolos seleksi jika memenuhi kriteria artikel berbobot:
        return is_article and (quality_score >= 0.70) and cpt_ready
```

7.8 Rekomendasi 8: Arsitektur Terpadu – The Agentic CPT Harvester Pipeline

Dengan memadukan **Hermes Agent** di hulu, **AITF Engine Mas Anjas** di tengah, dan **JEV TypeSafe AI** di hilir, sistem bertransformasi menjadi **Pipa Pemanen Data Otonom Generasi Baru**:

Pipa Pemanen Data CPT Otonom (Hermes + AITF + JEV) **1. FASE STRATEGI:** Hermes merumuskan 50 query Google dorking berdasarkan tema target.
2. FASE DISCOVERY: `aitf-engine` merayapi Google via Xvfb paralel (panen 500 URL).
3. FASE RESOLUSI: `httpx` mengurai redirect 302 Google ke URL portal asli dalam 5 detik.
4. FASE EKSTRAKSI: Async Playwright (8 worker) mengekstrak teks HTML artikel.
5. FASE KURASI MUTU: JEV System One memvalidasi tipe halaman dan skor mutu CPT.
6. FASE ARSIP: Teks yang lolos verifikasi JEV ditulis ke `dataset_cpt_verified.md`.

8 Panduan Praktis Pengoperasian End-to-End Saat Ini

Berikut adalah panduan langkah demi langkah menjalankan sistem AITF Crawler Engine yang ada saat ini di lingkungan Windows maupun Linux:

8.1 Langkah di Windows

1. **Persiapan Kata Kunci:** Buka berkas `Windows/daftar_keyword.txt` dan tuliskan topik pencarian (satu topik per baris):

```
Sekolah Rakyat
Matematika Aljabar
Sejarah Indonesia
```

2. **Jalankan Tahap 1 (Google Crawling):** Buka PowerShell di dalam folder `Crawler Mas Anjas/Windows`:

```
.\aitf-engine.exe crawl
   --token=R8trcqTXSWyPPnkIbOI6QKV3dm+2xsGeP90Eijj8AR9C1FyeRa6vgmzEx/sv3wravjgAH
```

   Tiga jendela Chrome akan terbuka secara otomatis dan merayapi Google. Tunggu hingga proses selesai dan berkas `Windows/output/crawler_url.txt` terisi.

3. **Jalankan Tahap 2 (Ekstraksi Konten Teks):** Buka terminal di root direktori project:

```
python Scrapper/scraper.py --urls=Windows/output/crawler_url.txt
   --output=Scrapper/output/hasil_scrapping.md --output-mode=merge
```

4. **Ambil Hasil:** Buka berkas `Scrapper/output/hasil_scrapping.md`. Seluruh artikel teks bersih yang siap dipakai sebagai dataset training CPT sudah tersimpan rapi dengan pemisah `--`.

8.2 Langkah di Linux (Ubuntu / Server Tanpa Monitor)

```
cd Linux
# Jalankan installer untuk Chrome, ChromeDriver, dan Xvfb:
chmod +x linux-installer.sh
./linux-installer.sh
source ~/.bashrc

# Jalankan Tahap 1 (otomatis menggunakan xvfb virtual monitor):
aitf-engine crawl --token={YOUR_TOKEN}

# Jalankan Tahap 2:
cd ../Scrapper
pip install -r requirements.txt
python -m playwright install chromium
python scraper.py --urls=../Linux/output/crawler_url.txt
   --output=output/cpt_dataset.md --output-mode=merge
```

9 Kesimpulan

AITF Crawler Engine buatan Mas Anjas adalah alat yang sangat efektif dan terfokus untuk memecahkan masalah pengumpulan korpus data latih AI dari mesin pencari Google. Pendekatan pemisahan dua tahap (*Two-Stage Decoupled Pipeline*) berhasil memecahkan tantangan proteksi bot Google Search tanpa membebani proses ekstraksi isi artikel.

Dengan menerapkan rekomendasi rekayasa modern yang dipaparkan dalam dokumen ini—terutama paralelisasi asinkron Playwright dan algoritma pembersihan boilerplate otomatis—sistem ini dapat ditingkatkan kinerjanya menjadi pipa data (*data pipeline*) berkecepatan tinggi yang mampu memanen puluhan ribu artikel per jam dengan kualitas dataset yang terjaga kemurniannya.